

6. Almacenamiento local

6.1. Descripción general

Los navegadores proporcionan una serie de mecanismos para que las páginas puedan **almacenar información persistente** de manera local. Lógicamente, el código JavaScript puede almacenar información en **variables**, pero dicha información se **pierde** cuando se **abandona** o se **recarga** la página.

Existen muchas **situaciones** en las que es deseable contar con un **almacenamiento persistente** de este tipo:

- **Recordar las preferencias** de un sitio web, como el idioma o el tema.
- **Recordar las credenciales de usuario** para no tener que iniciar sesión cada vez que se accede al servicio.
- Guardar **datos** de aplicaciones para poder continuar con el trabajo en otro momento.

A continuación, veremos dos de los métodos más utilizados: las **cookies** y el **web storage** o almacenamiento clave-valor.



IMPORTANTE

El almacenamiento en navegador se realiza en la carpeta de **perfil de usuario** del navegador. Por tanto, si se utiliza un navegador o un equipo **distinto** o si se inicia sesión con **otra cuenta de usuario**, no se tendrá acceso a los datos guardados.

No obstante, los navegadores proporcionan a su vez mecanismos de **sincronización entre distintos dispositivos**, que permiten compartir perfiles de usuario y por tanto también los datos locales de navegador.

6.2. Cookies

Las **cookies** son el sistema de almacenamiento más antiguo y forman parte del protocolo HTTP. Son **cade-nas de caracteres** que pueden ser almacenadas en el navegador. Las **cookies** pueden ser **escritas** y **leídas** por los sitios web que las utilizan, y se almacenan a nivel de **dominio**.



IMPORTANTE

Un sitio web **solo tiene acceso** a las cookies definidas por **su dominio**: no es posible que un sitio web (**sitio1.com**) acceda a las cookies de otro sitio de distinto dominio (**sitio2.com**).

Las **cookies** tienen un **tamaño máximo** de **4 kB**, por lo que no son muy adecuadas para almacenar gran cantidad de información.

Las **cookies** pueden ser creadas de dos maneras:

- Por el **servidor**. La **autenticación** es el principal caso de uso: una vez que el servidor valida la identidad del usuario (mediante un formulario de *login*, por ejemplo) **adjunta a la respuesta una cookie** con los parámetros de autenticación. El navegador reenviará dicha **cookie** con las sucesivas peticiones al mismo servidor. De esta manera, el usuario no tiene que enviar usuario y contraseña cada vez que navegue por una página distinta dentro del mismo sitio.
- Por el **cliente**. La lógica de cliente puede crear **cookies** mediante **JavaScript**.

El acceso a las *cookies* desde JavaScript solo puede realizarse mediante `document.cookie`. `document.cookie` es una propiedad que tiene definido un *setter*, un tipo especial de acceso en JavaScript que permite que la asignación de valores se realice a través de una *función*. En concreto, una asignación a `document.cookie` no sobrescribe todas las *cookies*, sino que solamente *actualiza las cookies indicadas*. A continuación, se muestra un ejemplo:

```
// Supongamos que document.cookie = "cookie1=valor1; cookie2=valor2"

document.cookie = "cookie1=valorNuevo";
console.log(document.cookie); // "cookie1=valorNuevo; cookie2=valor2"

document.cookie = "cookie3=valor3";
console.log(document.cookie); // "cookie1=valorNuevo; cookie2=valor2;
cookie3=valor3;"
```

Las *cookies* pueden definir también *propiedades*, algunas de las cuales se indican a continuación:

- **max-age**. Tiempo de vida en segundos. Pasado el tiempo, la *cookie* se borra.
- **expires**. Fecha de caducidad en formato texto. Pasado el tiempo, la *cookie* se borra.
- **secure**. La *cookie* solo debe enviarse a través de HTTPS.

```
// Cookie con un tiempo de vida de una hora
document.cookie = "cookie1=valorNuevo; max-age=3600";
```

Dado que el uso de `document.cookie` es algo engorroso, se suelen utilizar *expresiones regulares* para obtener las *cookies* que se desean buscar.



EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Enlaces web*, puedes consultar *The Modern JavaScript Tutorial*, donde se muestran algunas funciones que utilizan expresiones regulares para trabajar con *cookies* de manera más sencilla.

6.3. API de Almacenamiento Web

Como hemos visto en el apartado anterior, las *cookies* son muy utilizadas para *interactuar con el servidor*, ya que están integradas en el protocolo HTTP [se reenvían con cada petición] y facilitan mucho tareas como la *autenticación*. Sin embargo, en JavaScript son *complicadas de utilizar*, y además tienen una capacidad de almacenamiento muy limitada. Para solucionar estos problemas, se creó la **API de Almacenamiento Web** o *Web Storage*.

Las *características* del almacenamiento web son las siguientes:

- Su contenido **no se envía** con cada petición.
- Tiene una capacidad de almacenamiento de **5 MB**.
- Solo es accesible a través de **JavaScript**.

Existen **dos tipos** de almacenamiento web:

- **localStorage**. Almacena datos hasta que se borren de manera **explícita**.
- **sessionStorage**. Almacena datos hasta que se cierre la pestaña del navegador.

Ambos tipos son sistemas de almacenamiento **clave-valor**. Ambos definen las siguientes **propiedades y métodos**:

- **setItem(clave, valor)**. Almacena el valor en la clave correspondiente. También se puede utilizar notación de objetos, como, por ejemplo, `localStorage.clave = "valor"`.
- **getItem(clave)**. Obtiene el valor de la clave indicada. También se puede utilizar notación de objetos, como, por ejemplo, `localStorage.clave`.
- **removeItem(clave)**. Elimina la clave indicada.
- **clear()**. Borra todas las claves.
- **length**. Devuelve el número de claves almacenadas.

Tanto las **claves** como los **valores** son de tipo **string**. Si se desea almacenar otro tipo de datos (objetos o arrays, por ejemplo) se puede utilizar **JSON**.

! IMPORTANTE

Es importante saber que si se quieren guardar objetos **que tengan métodos**, dichos métodos **no se exportan a JSON**. Por tanto, si se quiere guardar y posteriormente recuperar un objeto del almacenamiento local, **habrá que reconstruir la funcionalidad del objeto** una vez cargado.

Este proceso se denomina **rehidratación** y puede realizarse creando un nuevo objeto a partir de su **prototipo** (para tener acceso a los métodos) y **Object.assign** (para copiar las **propiedades de datos** guardadas en el almacenamiento web).

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo **DWEC_U03_A06_01.html**, con el código de ejemplo.

** El archivo indicado en el ejemplo anterior es un archivo con código JavaScript; se debería renombrar a **.js**, o bien habría que incluir código HTML para que se pueda abrir en el navegador.